

AI Resume Screening & Ranking System

SDE Intern Coding Assignment - Technical Assessment Report

Date: October 09, 2026

Author: Aritra Das

Executive Summary

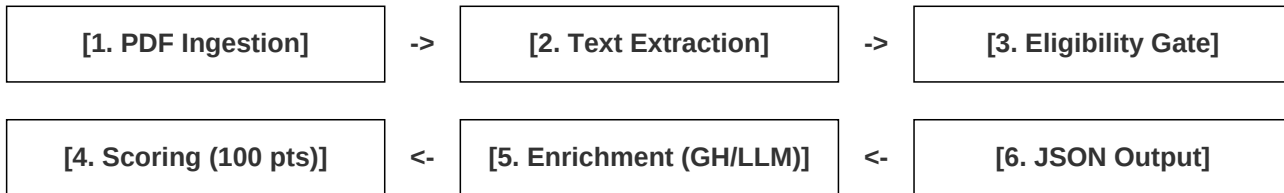
This report documents the architecture, methodology, and results of the AI Resume Screening & Ranking System. The system automates the evaluation of software engineering candidate resumes against strict criteria: demonstrated Python proficiency and substantive AI/RAG/Agentic project experience. It implements a robust, deterministic pipeline that ingests resumes, extracts critical information, filters candidates based on hard requirements, scores them against a 100-point rubric, and enriches the data using the GitHub API and LLM-assisted semantic analysis.

Technologies utilized: Python 3.12, PyMuPDF (Ingestion), Regex/Heuristics (Extraction), GitHub REST API (Enrichment), Google Gemini / OpenAI (LLM Assessment), Streamlit (Frontend Dashboard), and Pytest (Validation).

Problem, Requirements, and Architecture

The core problem requires screening a batch of 50 resumes to reliably identify candidates capable of building production AI pipelines. It demands a defensible, bias-resistant methodology where mere keyword stuffing does not equate to engineering competence. The system must process PDFs, rank candidates mathematically, and provide explainable evidence for every score.

The implemented architecture strictly enforces a modular, linear workflow:



1. Ingestion: PyMuPDF extracts text directly from PDF bytes, handling unreadable files gracefully without crashing the batch.
2. Extraction: Heuristic filters strip boilerplate to extract candidate names, emails, and GitHub URLs.
3. Eligibility: A hard deterministic gate requiring both Python and AI evidence.
4. Scoring: A 100-point evaluator measuring project context rather than just keyword presence.
5. GitHub & LLM: Optional enrichment APIs that add qualitative depth and activity signals, designed with strict timeout fallbacks.

Eligibility and Ranking Methodology

Eligibility is strictly deterministic. A candidate must demonstrate both genuine Python evidence and meaningful AI/LLM/RAG/agentive evidence. This gate is not controlled by an LLM to prevent hallucinations from advancing unqualified candidates. Short acronyms like 'RAG' use word-boundary regular expressions to prevent false positives (e.g., matching inside the word 'storage').

Scoring Weights (100 Points Maximum)

- AI/Agentive/RAG Project Depth: 40 points
- Python/Backend Engineering: 30 points
- Cloud/Deployment/Full Stack: 15 points
- GitHub Activity (API Enriched): 10 points
- Engineering Depth (Testing, CI/CD): 5 points

Scoring heavily favors keywords found in contextual proximity to action verbs (e.g., 'built', 'deployed') over long lists of isolated skills. To combat inflated resumes, a 5-15 point penalty is applied when projects demonstrate 'thin wrapper' characteristics such as basic API calls, tutorial-style apps, or when AI keywords only appear in the skills section without project implementation details.

GitHub enrichment acts as a bonus signal. If a profile is private, missing, or the API rate limit is reached, the candidate safely scores 0 in this category without failing the screening.

Implementation, Reliability, and Testing

The implementation separates configuration (weights/keywords) from business logic. The Phase 2 LLM assessment integrates via a configurable adapter supporting Google Gemini and OpenAI. It uses strict JSON-schema prompting to perform semantic extraction, identifying true engineering strengths and confirming thin-wrapper penalties. Crucially, if the LLM API key is missing or the response times out, the pipeline deterministically falls back to regex-based scoring, guaranteeing reliability.

The system includes a Streamlit dashboard (app.py) that acts as a UI overlay for the core pipeline, ensuring no logic duplication.

Test Strategy (53 Passing Tests)

The test suite relies on pytest and uses synthetic resume strings to isolate behaviors:

- Eligibility limits: Asserts that Python-only or AI-only resumes fail the hard filter.
- Keyword boundaries: Prevents substrings from triggering false evidence.
- Mathematical bounds: Asserts no candidate can exceed category maximums or 100 total points.
- API Mocks: Simulates GitHub 403 Rate Limits, 404s, and LLM key absences to verify graceful fallback.

Results and Evaluation

The pipeline was executed against the provided 50-resume dataset. The actual processed batch metrics are as follows:

Total Resumes:	50
Parsed:	50
Eligible:	39
Rejected:	11
Failed:	0

Top 5 Ranked Candidates

Rank	Name	Score	Breakdown (AI/Py/Cld/GH/Eng)
1	Yash Maini	92	40/30/13/7/2
2	ABHINAV MISHRA	90	39/22/15/9/5
3	Arjun Kumar	89	40/30/10/7/2
4	V Sree Raghu Vardhan	89	40/30/14/0/5
5	Sumaiya Sultana Shaik	88	40/28/15/0/5

The score distribution demonstrates effective separation between candidates with deep, contextualized agentic implementations and those merely listing generic frameworks.

Design Decisions & Limitations

Trade-off: Determinism vs Semantic Understanding. A purely LLM-driven pipeline risks inconsistent eligibility judgments due to non-determinism. By isolating the LLM as an optional enrichment layer (Phase 2), the system retains absolute baseline reliability. If the LLM flags a 'thin wrapper', it is logged as a qualitative concern rather than silently mutating the quantitative score, maintaining auditability.

Known Limitations

- Name Extraction: Relies on heuristics (rejecting common non-name words and headers). Highly unconventional formats may still produce errors.
- API Reliance: GitHub restricts unauthenticated IPs to 60 requests per hour. For 50 resumes, a token is highly recommended to avoid partial enrichment.
- LLM Context Windows: Extremely dense, multi-page PDFs are truncated to ~3000 characters before LLM assessment to prevent token limit errors.

If I Had More Time (Future Improvements)

- Async/Concurrent Processing: Parallelizing I/O-bound tasks (GitHub and LLM API requests) to process the batch in seconds rather than minutes.
- Section-Aware Parsing: Using an ML classifier to segment the resume (Experience, Projects, Education) to isolate skill searches specifically to professional experience.
- Confidence Scoring: Outputting an algorithmic confidence percentage for each candidate to automatically flag ambiguous resumes for human review.